

0.3.2 状态依赖自适应协同（SDAS）模型的软件实现

本节基于第??节建立的 SDAS 理论框架，系统地介绍该模型在 Python 编程环境中的完整软件实现。在进入具体实现细节之前，本节首先概述软件系统的整体架构与数据流。

软件架构与数据流

本文开发的折纸手仿真工具包 `synergy_hand_sim` 在功能上涵盖从 CAD 设计到交互式三维物理仿真的完整流程。就 SDAS 模型而言，其核心计算链路由以下六个功能层次组成：

1. 设计层（CAD 编辑器模块）：`src/origami_cad/` 包基于 PyQt5 实现了图形化的折纸手设计编辑器。用户通过该编辑器在二维画布上绘制折痕线（区分峰折/谷折/轮廓）、添加面片、放置滑轮和孔类导向元件、定义缆绳路径与驱动器属性。编辑器内置了面片拓扑自动重建和设计验证功能。完成设计后，用户可将所有几何与拓扑信息保存为 `.ohd` 文件（JSON 格式），该文件是连接设计与仿真的桥梁。
2. 数据模型层：以 `OrigamiHandDesign` 类为核心容器，管理折痕线（`fold_lines`）、面片（`faces`）、滑轮（`pulleys`）、孔（`holes`）、腱绳路径（`tendons`）和驱动器（`actuators`）等全部设计数据。该层仅负责存储和查询，不涉及任何算法逻辑。
3. 传动计算层：`TransmissionBuilder` 模块实现 `compute_R_one_sided()` 函数，从 `OrigamiHandDesign` 中提取单侧 Capstan 衰减传动向量 $\mathbf{R}_A$ 和 $\mathbf{R}_B$ 。
4. 协同求解层：`SDASModel` 类封装了第??节中的 Schur 补公式和单向公式。在构造阶段接收传动向量 $\mathbf{R}_A$ 和 $\mathbf{R}_B$ ，预计算所有系数矩阵；在运行阶段接收电机位移输入，输出满足传动约束的关节角度向量。
5. 仿真层：包含两套并行的仿真机制。运动学交互仿真器（`MuJoCoSimulator`）基于 MuJoCo 物理引擎，通过滑块回调接口实时驱动三维模型；动力学数值仿真器（`HandSimulator`）包含惯性与阻尼模型，通过 ODE 积分输出关节角度的时间历程。
6. 可视化层：MuJoCo 原生 GLFW 查看器提供三维渲染，滑块面板提供交互控制；动力学仿真和运动学仿真的结果还可通过 MeshCat 浏览器端可视化或 `SimulationVisualizer` 生成多面板图表。

图1展示了上述模块之间完整的数据流关系。系统以 Origami CAD 编辑器为设计入口，所有几何和拓扑信息通过 `.ohd` 文件传递至数据模型层，随后分流至 URDF 导出和传动矩阵计算两条路径，最终汇聚于 MuJoCo 交互式仿真器和动力学数值仿真器。

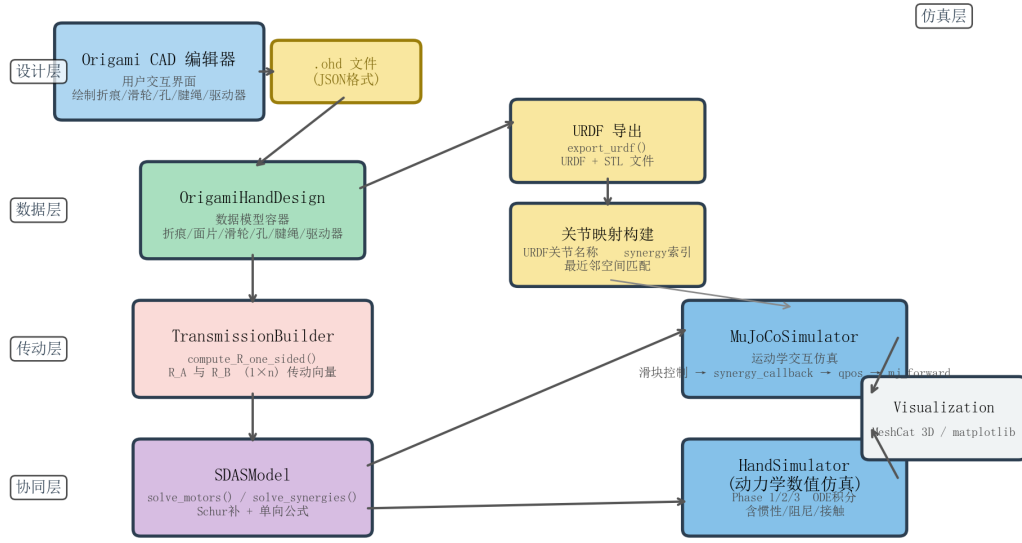


Figure 1: SDAS 模型软件系统架构与数据流。方框代表功能模块，箭头表示数据流向。左侧纵向排列了从设计层到协同层的四层结构，右侧为仿真层与可视化层。

以下 Python 脚本可生成图 1 所示的系统架构图。该脚本基于 matplotlib 库，通过 FancyBboxPatch 绘制圆角矩形模块，使用 annotate 绘制带箭头的连线标注数据流。运行时生成 architecture_diagram.png 文件。

```

1 import matplotlib.pyplot as plt
2 import matplotlib.patches as mpatches
3
4 def draw_box(ax, center, width, height, text, subtext='',
5             facecolor='#D6EAF8', edgecolor='#2C3E50'):
6     x, y = center
7     rect = mpatches.FancyBboxPatch(
8         (x - width/2, y - height/2), width, height,
9         boxstyle="round,pad=0.1",
10        facecolor=facecolor, edgecolor=edgecolor, lw=2)
11    ax.add_patch(rect)
12    ax.text(x, y + height*0.15, text, ha='center', va='center',
13           fontsize=10, fontweight='bold')
14    if subtext:
15        ax.text(x, y - height*0.20, subtext, ha='center', va='center'

```

```

16         fontsize=8, color='#444444')
17
18 def draw_io_box(ax, center, width, height, text,
19               facecolor='#F9E79F', edgecolor='#9A7D0A'):
20     x, y = center
21     rect = mpatches.FancyBboxPatch(
22         (x - width/2, y - height/2), width, height,
23         boxstyle="round,pad=0.1",
24         facecolor=facecolor, edgecolor=edgecolor, lw=2)
25     ax.add_patch(rect)
26     ax.text(x, y, text, ha='center', va='center',
27            fontsize=9, fontweight='bold', color='#7D6608')
28
29 fig, ax = plt.subplots(figsize=(12, 8))
30 ax.set_xlim(-1, 13); ax.set_ylim(-1, 9); ax.axis('off')
31 draw_box(ax, (1.0, 7.0), 2.6, 1.2, 'Origami CAD Editor',
32         'User interface\nfolds/pulleys/holes/tendon paths',
33         facecolor='#AED6F1')
34 draw_io_box(ax, (3.5, 7.0), 1.8, 0.7, '.ohd File\n(JSON)')
35 draw_box(ax, (2.25, 5.0), 2.8, 1.2, 'OrigamiHandDesign',
36         'Data container\nfolds/faces/pulleys/tendons',
37         facecolor='#A9DFBF')
38 draw_box(ax, (2.25, 3.0), 3.0, 1.2, 'TransmissionBuilder',
39         'compute_R_one_sided()\nR_A, R_B vectors',
40         facecolor='#FADBD8')
41 draw_box(ax, (2.25, 1.0), 3.0, 1.2, 'SDASModel',
42         'solve_motors() / solve_synergies()\nSchur complement',
43         facecolor='#D7BDE2')
44 draw_box(ax, (7.0, 6.0), 2.4, 1.0, 'URDF Export',
45         'export_urdf()\nURDF + STL files',
46         facecolor='#F9E79F')
47 draw_box(ax, (7.0, 4.5), 2.4, 1.0, 'Joint Mapping',
48         'URDF joint name to\nsynergy index mapping',
49         facecolor='#F9E79F')
50 draw_box(ax, (9.5, 3.0), 2.8, 1.2, 'MuJoCoSimulator',
51         'Interactive simulation\nslider control',
52         facecolor='#85C1E9')
53 draw_box(ax, (9.5, 1.0), 2.8, 1.2, 'HandSimulator',
54         'Numerical dynamics\nODE integration',
55         facecolor='#85C1E9')

```

```

56 draw_box(ax, (11.5, 2.0), 2.0, 1.0, 'Visualization',
57          'MeshCat / matplotlib',
58          facecolor='#F0F3F4')
59 # Connect boxes with arrows (annotate)
60 for (sx, sy), (ex, ey) in [
61     ((2.3, 7.0), (2.6, 7.0)), # CAD -> .ohd
62     ((3.5, 6.65), (3.5, 5.6)), # .ohd -> OrigamiHandDesign
63     ((3.65, 5.3), (5.8, 6.0)), # OrigamiHandDesign -> URDF
64     ((2.25, 4.4), (2.25, 3.6)), # OrigamiHandDesign ->
        TransmissionBuilder
65     ((2.25, 2.4), (2.25, 1.6)), # TransmissionBuilder -> SDAS
66     ((7.0, 5.5), (7.0, 5.0)), # URDF -> Joint Mapping
67     ((3.75, 1.3), (8.1, 3.3)), # SDAS -> MuJoCo
68     ((3.75, 0.7), (8.1, 1.2)), # SDAS -> HandSimulator
69     ((10.9, 3.3), (10.5, 2.5)), # MuJoCo -> Visualization
70     ((10.9, 1.3), (10.5, 2.0)), # HandSimulator -> Visualization
71 ]:
72     ax.annotate('', xy=(ex, ey), xytext=(sx, sy),
73               arrowprops=dict(arrowstyle='->', color='#555555', lw
74                               =1.5))
75 plt.title('SDAS Software System Architecture', fontsize=14,
76          fontweight='bold')
77 plt.savefig('architecture_diagram.png', dpi=200, bbox_inches='tight')

```

Listing 1: 系统架构图生成脚本

运行该脚本即可生成图 1 所示的系统架构图。

核心数据结构：OrigamiHandDesign

SDAS 模型软件实现的数据基础是 `OrigamiHandDesign` 类(见 `src/models/origami_design.py`)。该类的设计遵循“数据与算法分离”原则——类本身仅负责存储和查询，不涉及传动矩阵计算或运动学求解等算法逻辑。与 SDAS 模型密切相关的成员字段及其含义列于表 1。

关节索引的获取通过 `get_joint_list()` 函数完成。该函数位于 `src/models/transmission_builder.py`。其返回值包含两个元素：按 ID 排序的 `JointConnection` 列表，以及 `fold_line_id` 到关节索引的映射字典。当设计尚未重建拓扑时，函数直接从 `fold_lines` 中筛选谷折和山折类型的折痕线作为关节，从而保留原始的 `fold_line_ID`，使滑轮和孔中记录的 `attached_fold_line_id` 与关节索引之间的映射关系不断裂。

从 SDAS 模型的角度来看，腱绳路径的拓扑结构是最关键的信息。以表 1 中的 `tendons` 字段为例，一条包含元素序列 $[e_0, e_1, \dots, e_{N-1}]$ 的腱绳中， $e_0 = -1$ (Motor A)、 $e_{N-1} = -2$ (Motor B)，中间元素为滑轮或孔。该序列唯一确定了从两台驱动器出发的

Table 1: OrigamiHandDesign 中与 SDAS 模型密切相关的关键属性

字段	数据类型	说明
fold_lines	dict[int, FoldLine]	折痕字典。每个 FoldLine 包含起止点坐标、折痕类型 (MOUNTAIN/VALLEY/OUTLINE) 和弯曲刚度系数 κ_i 。每个活动关节对应一条折痕线。
faces	dict[int, OrigamiFace]	面片字典。每个面片由顶点序列和边 ID 序列定义，用于 URDF 导出和碰撞检测网格生成。
joints	list[JointConnection]	关节连接表。每条记录包含关联的折痕线 ID、两个面片 ID 和折痕类型，描述相邻面片间的转动自由度。
pulleys	dict[int, Pulley]	滑轮字典。滑轮 ID 为正整数，包含位置、半径 r 、摩擦系数 μ 和附着折痕 ID。
holes	dict[int, Hole]	孔字典。孔 ID ≤ -100 ，包含位置、板厚偏移 h 、摩擦系数 μ 和附着折痕 ID。
tendons	dict[int, TendonPath]	腱绳路径字典。记录元素 ID 序列：正数为滑轮，-1 为驱动器 A，-2 为驱动器 B， ≤ -100 为孔， ≤ -200 为阻尼器。
actuators	dict[int, Actuator]	驱动器字典。预定义两个驱动器，ID 分别为 -1 (Motor A) 和 -2 (Motor B)。
dampers	dict[int, Damper]	阻尼器字典。用于动态协同模型，SDAS 模型不使用该字段。

Capstan 张力衰减路径，从而决定了 $\mathbf{R}_A$ 与 $\mathbf{R}_B$ 两个传动向量的分布。`tendons` 中的元素 ID 编码约定由 CAD 编辑器自动维护，确保解析过程的安全性。

单侧传动矩阵 $\mathbf{R}_A$ 与 $\mathbf{R}_B$ 的计算

传动矩阵 $\mathbf{R}_A$ 和 $\mathbf{R}_B$ 的计算是 SDAS 模型区别于经典自适应协同模型的核心环节。在经典自适应协同中，传动矩阵 $\mathbf{R}$ 被视为固定常数矩阵，物理上隐含了腱绳两端张力相等同时作用的假设。然而，真实的绳驱系统中，由于 Capstan 摩擦使张力沿路径指数衰减，两端驱动器的张力分布并不相同。

物理模型回顾 由第 ?? 节中的式 (2.23) 和 (2.25) 可知，当驱动器 A 输入张力 F_A 时，传播至第 j 个导向元件时的衰减因子为 $\alpha_j^{(A)} = \exp(-\sum_{k \in \mathcal{P}_A(j)} \beta_k)$ ，其中 $\beta_k = \mu_k \theta_k$ 为第 k 个导向元件的有效摩擦系数。类似地，从驱动器 B 出发有 $\alpha_j^{(B)} = \exp(-\sum_{k \in \mathcal{P}_B(j)} \beta_k)$ 。在实用中取所有 $\beta_k \equiv \beta$ ，则对于第 j 个元素有 $\alpha_j^{(A)} = e^{-\beta j}$ 和 $\alpha_j^{(B)} = e^{-\beta(N-1-j)}$ ，其中 N 为路径元素总数。

参数 β 的默认值为 $\beta = 0.09$ 。该取值的物理含义是：单个导向元件导致的张力衰减约为 $1 - e^{-0.09} \approx 8.6\%$ 。经过 42 个导向元件后，张力衰减至初始值的 $e^{-0.09 \times 42} \approx 0.023$ ，即约 2.3%。用户也可以在 CAD 编辑器中为每个滑轮单独设定摩擦系数 μ_k ，此时 β_k 由 μ_k 与默认包角 θ_k 的乘积确定。

由第 ?? 节中的式 (2.30) 可知， $\mathbf{R}_A$ 和 $\mathbf{R}_B$ 的第 i 个分量为附着于关节 i 的所有导向

元件的半径 r_k 经衰减加权后的累加和:

$$R_A[i] = \sum_{k \in \mathcal{J}_i} r_k \cdot e^{-\beta \cdot d_A(k)}, \quad (1)$$

$$R_B[i] = \sum_{k \in \mathcal{J}_i} r_k \cdot e^{-\beta \cdot d_B(k)}, \quad (2)$$

其中 $\mathcal{J}_i$ 为附着于关节 i 的所有导向元件集合, $d_A(k)$ 和 $d_B(k)$ 分别为元素 k 到驱动器 A 和 B 沿路径的步数。对于孔类导向元件, r_k 取 `plate_offset` (板厚偏移) 而非几何半径, 这是因为孔的等效驱动力臂由纸张折叠后孔中心到折痕轴线的垂直距离决定。

伪代码描述 函数 `compute_R_one_sided()` 实现了上述计算。算法 1 给出了完整伪代码。

Algorithm 1 单侧传动矩阵 $\mathbf{R}_A$ 与 $\mathbf{R}_B$ 的计算

Require: origami design $design$, friction coefficient β

Ensure: transmission vectors $\mathbf{R}_A \in \mathbb{R}^{1 \times n}$, $\mathbf{R}_B \in \mathbb{R}^{1 \times n}$

```

1:  $joint\_list, id\_map \leftarrow extract\_joints(design)$ 
2:  $\mathcal{R}_A \leftarrow []$ ;  $\mathcal{R}_B \leftarrow []$ 
3: for each tendon  $t \in design.tendons$  do
4:    $\mathcal{E} \leftarrow filter\_elements(t.pulley\_sequence)$ 
5:    $\tilde{R}_A \leftarrow \mathbf{0}$ ;  $\tilde{R}_B \leftarrow \mathbf{0}$ 
6:   for  $j \leftarrow 0$  to  $N - 1$  do
7:      $e_j \leftarrow \mathcal{E}[j]$ 
8:      $r \leftarrow get\_element\_radius(e_j, design)$ 
9:     if  $r \leq 0$  then continue
10:    end if
11:     $i \leftarrow get\_element\_joint\_idx(e_j, design, id\_map)$ 
12:    if  $i$  is valid then
13:       $d_A \leftarrow j$ ;  $d_B \leftarrow N - 1 - j$ 
14:       $\tilde{R}_A[i] \leftarrow \tilde{R}_A[i] + r \cdot e^{-\beta d_A}$ 
15:       $\tilde{R}_B[i] \leftarrow \tilde{R}_B[i] + r \cdot e^{-\beta d_B}$ 
16:    end if
17:  end for
18:   $\mathcal{R}_A.append(\tilde{R}_A)$ ;  $\mathcal{R}_B.append(\tilde{R}_B)$ 
19: end for
20:  $\mathbf{R}_A \leftarrow mean(\mathcal{R}_A, axis = 0)$ 
21:  $\mathbf{R}_B \leftarrow mean(\mathcal{R}_B, axis = 0)$ 
22: return ( $\mathbf{R}_A, \mathbf{R}_B$ )
```

计算中有两点需说明。第一, `get_element_radius()` 对滑轮返回其 `radius` 字段, 对孔则返回 `plate_offset` 字段——孔的等效半径由折纸板的厚度决定。第二, 当存在多条驱动腱绳时, 取所有驱动腱绳的算术平均作为最终的 $\mathbf{R}_A$ 和 $\mathbf{R}_B$, 这是因为所有腱绳共享相同的驱动器 A 和 B, 平均传动向量共同决定系统的宏观传动行为。算法 1 的

计算复杂度为 $\mathcal{O}(N_t \cdot N_e)$ ，其中 N_t 为腱绳数量， N_e 为每条腱绳的导向元件数量。典型的三指折纸手设计（1 条驱动腱绳、约 40 个导向元件）单次计算耗时约 0.1 ms。

SDAS 模型类的实现

SDAS 的约束求解器封装在 `SDASModel` 类中（见 `src/synergy/sdas_model.py`）。构造函数接收四个参数：关节数量 n 、行向量 $\mathbf{R}_A \in \mathbb{R}^{1 \times n}$ 、行向量 $\mathbf{R}_B \in \mathbb{R}^{1 \times n}$ 以及关节刚度向量 $\mathbf{E}_{\text{vec}} \in \mathbb{R}^n$ 。

预计算 在构造阶段，首先建立对角刚度矩阵 $\mathbf{E} = \text{diag}(\mathbf{E}_{\text{vec}})$ 及其逆 $\mathbf{E}^{-1} = \text{diag}(1/\mathbf{E}_{\text{vec}})$ 。

由第 ?? 节中的式 (2.40)–(2.41) 可知，Schur 补求解需要先计算三个标量系数：

$$\begin{aligned} a &= \mathbf{R}_A \mathbf{E}^{-1} \mathbf{R}_A^\top, \\ b &= \mathbf{R}_A \mathbf{E}^{-1} \mathbf{R}_B^\top, \\ c &= \mathbf{R}_B \mathbf{E}^{-1} \mathbf{R}_B^\top, \end{aligned} \tag{3}$$

$$\Delta = ac - b^2. \tag{4}$$

当 $\Delta < 10^{-15}$ 时系统退化，此时 $\mathbf{R}_A$ 与 $\mathbf{R}_B$ 线性相关，通常出现在对称路径且刚度均匀的条件下。构造函数中对此进行检查并给出警告。

随后由第 ?? 节中的式 (2.39) 提取出 $\mathbf{S}_A$ 和 $\mathbf{S}_B$ 的系数向量：

$$\mathbf{S}_A^{(\text{schur})} = \mathbf{E}^{-1} (c \mathbf{R}_A^\top - b \mathbf{R}_B^\top) / \Delta, \tag{5}$$

$$\mathbf{S}_B^{(\text{schur})} = \mathbf{E}^{-1} (-b \mathbf{R}_A^\top + a \mathbf{R}_B^\top) / \Delta. \tag{6}$$

同步预计算单马达单向公式的系数向量：

$$\mathbf{S}_A^{(\text{paper})} = \mathbf{E}^{-1} \mathbf{R}_A^\top / a, \tag{7}$$

$$\mathbf{S}_B^{(\text{paper})} = \mathbf{E}^{-1} \mathbf{R}_B^\top / c. \tag{8}$$

以及外力柔顺矩阵 $\mathbf{C}$ 。

伪代码描述 算法 2 和算法 3 分别给出了构造函数和核心求解方法的伪代码。

`solve_synergies()` 是高层接口，接收协同变量 σ （共模分量）和 σ_f （差模分量）作为输入。由第 ?? 节中电机位移与协同变量的关系可知，二者满足：

$$\theta_A = \max(0, \sigma + \sigma_f), \quad \theta_B = \max(0, \sigma - \sigma_f), \tag{9}$$

其中 $\sigma \geq 0$ 。由于 `solve_motors()` 内部已包含松弛钳位，当 $|\sigma_f| > \sigma$ 时 θ_B 或 θ_A 被自动钳位为零，系统退化为单马达模式。这种切换准确反映了物理世界中腱绳松弛侧不再

Algorithm 2 SDASModel 构造函数的预计算逻辑

Require: joint count n , transmission vectors $\mathbf{R}_A, \mathbf{R}_B$, stiffness vector $\mathbf{E}_{\text{vec}}$

```
1: Store  $\mathbf{R}_A, \mathbf{R}_B, \mathbf{E}_{\text{vec}}$ 
2:  $\mathbf{E} \leftarrow \text{diag}(\mathbf{E}_{\text{vec}}); \mathbf{E}^{-1} \leftarrow \text{diag}(1/\mathbf{E}_{\text{vec}})$ 
3:  $a \leftarrow \mathbf{R}_A \mathbf{E}^{-1} \mathbf{R}_A^\top$ 
4:  $b \leftarrow \mathbf{R}_A \mathbf{E}^{-1} \mathbf{R}_B^\top$ 
5:  $c \leftarrow \mathbf{R}_B \mathbf{E}^{-1} \mathbf{R}_B^\top$ 
6:  $\Delta \leftarrow ac - b^2$ 
7: if  $\Delta < 10^{-15}$  then
8:   Warning:  $\mathbf{R}_A$  and  $\mathbf{R}_B$  are nearly linearly dependent
9: end if
10:  $\mathbf{S}_A^{(\text{schur})} \leftarrow \mathbf{E}^{-1}(c\mathbf{R}_A^\top - b\mathbf{R}_B^\top)/\Delta$ 
11:  $\mathbf{S}_B^{(\text{schur})} \leftarrow \mathbf{E}^{-1}(-b\mathbf{R}_A^\top + a\mathbf{R}_B^\top)/\Delta$ 
12:  $\mathbf{S}_A^{(\text{paper})} \leftarrow \mathbf{E}^{-1}\mathbf{R}_A^\top/a$ 
13:  $\mathbf{S}_B^{(\text{paper})} \leftarrow \mathbf{E}^{-1}\mathbf{R}_B^\top/c$ 
14:  $\mathbf{S}_\sigma^{(\text{schur})} \leftarrow \mathbf{S}_A^{(\text{schur})} + \mathbf{S}_B^{(\text{schur})}$ 
15:  $\mathbf{S}_{\text{diff}}^{(\text{schur})} \leftarrow \mathbf{S}_A^{(\text{schur})} - \mathbf{S}_B^{(\text{schur})}$ 
16: return
```

Algorithm 3 solve_motors() 的约束选择逻辑

Require: motor displacements θ_A, θ_B (rad), threshold $\varepsilon = 10^{-10}$

Require: optional: Jacobian $\mathbf{J}$, external force $\mathbf{f}_{\text{ext}}$

Ensure: joint angle vector $\mathbf{q} \in \mathbb{R}^n$ $\triangleright$ Slack clamp: tendon can only transmit tensile force

```
1:  $\theta_A \leftarrow \max(0, \theta_A); \theta_B \leftarrow \max(0, \theta_B)$   $\triangleright$  State-dependent constraint selection
2: if  $\theta_A > \varepsilon$  and  $\theta_B > \varepsilon$  then  $\triangleright$  Both motors active: Schur complement formula
3:    $\mathbf{q} \leftarrow \mathbf{S}_A^{(\text{schur})} \cdot \theta_A + \mathbf{S}_B^{(\text{schur})} \cdot \theta_B$ 
4: else if  $\theta_A > \varepsilon$  then  $\triangleright$  Only Motor A active: one-sided formula
5:    $\mathbf{q} \leftarrow \mathbf{S}_A^{(\text{paper})} \cdot \theta_A$ 
6: else if  $\theta_B > \varepsilon$  then  $\triangleright$  Only Motor B active: one-sided formula
7:    $\mathbf{q} \leftarrow \mathbf{S}_B^{(\text{paper})} \cdot \theta_B$ 
8: else  $\triangleright$  Both slack: zero output
9:    $\mathbf{q} \leftarrow \mathbf{0} \in \mathbb{R}^n$ 
10: end if
11: if external force information exists and is non-zero then
12:    $\mathbf{q} \leftarrow \mathbf{q} + \mathbf{C} \cdot \mathbf{J}^\top \mathbf{f}_{\text{ext}}$ 
13: end if return  $\mathbf{q}$ 
```

传递驱动力的行为。

MuJoCo 交互仿真器集成

交互式仿真环境是检验 SDAS 模型实时性能和验证行为合理性的重要工具。系统通过 `scripts/run_synergy_from_ohd.py` 脚本启动，其工作流程分为构建与运行两个阶段。

构建阶段 程序启动时依次执行以下步骤：(i) 通过 `OrigamiHandDesign.load()` 加载 `.ohd` 文件；(ii) 调用 `build_sdas_model()` 构建 SDAS 模型——其间依次执行关节提取、 $\mathbf{R}_A/\mathbf{R}_B$ 计算和 `SDASModel` 构造；(iii) 通过空间最近邻匹配建立 URDF 关节名称与协同模型关节索引的映射关系——该算法将每个 URDF 铰链关节的全局坐标与所有折痕线中点进行距离比对，取最近匹配；(iv) 启动 `MuJoCoSimulator`，传入协同回调函数 `synergy_callback` 和 URDF 到 Synergy 的映射字典。

运行时的滑块回调机制 MuJoCo 仿真器右侧面板包含四个滑块，分别对应 Motor A 位移、Motor B 位移、共模协同变量 σ 和差模协同变量 σ_f 。滑块之间具有互联动功能，由以下逻辑实现：

1. 比较当前帧 `data.ctrl` 与上一帧 `prev_ctrl` 的差值。若 A/B 滑块变化量主导，则通过式 (9) 的逆映射 $\sigma = (\theta_A + \theta_B)/2$ 、 $\sigma_f = (\theta_A - \theta_B)/2$ 更新 σ/σ_f 滑块。
2. 若 σ/σ_f 滑块变化量主导，则通过式 (9) 的正向映射计算 θ_A/θ_B 并更新 A/B 滑块。
3. 若两种变化量均小于容差，保持上次互联动主从关系不变。

算法 4 给出了协同回调函数的伪代码。

Algorithm 4 MuJoCo 仿真器 `synergy_callback` 回调函数

Require: slider values θ_1, θ_2 (Motor A, Motor B)

Require: `SDASModel`, URDF-to-synergy mapping M

Ensure: dict $\mathcal{Q} = \{\text{URDF joint name} \rightarrow \text{clamped angle}\}$ ▷ Step 1: Slack clamp

- 1: $\theta_A \leftarrow \max(0.0, \theta_1)$; $\theta_B \leftarrow \max(0.0, \theta_2)$ ▷ Step 2: SDAS solve
- 2: $\mathbf{q} \leftarrow \text{sdas_model.solve_motors}(\theta_A, \theta_B)$ ▷ Step 3: Joint limit clamp
- 3: $\mathcal{Q} \leftarrow \{\}$
- 4: **for** each mapping pair $(k, i) \in M$ **do**
- 5: **if** $0 \leq i < \text{len}(\mathbf{q})$ **then**
- 6: $\theta_{\text{raw}} \leftarrow \mathbf{q}[i]$
- 7: $\theta_{\text{clamp}} \leftarrow \text{clamp}(\theta_{\text{raw}}, 0, \pi)$ ▷ valley **or** $\text{clamp}(\theta_{\text{raw}}, -\pi, 0)$ ▷ mountain
- 8: $\mathcal{Q}[k] \leftarrow \theta_{\text{clamp}}$
- 9: **end if**
- 10: **end for** ▷ Step 4: Output to MuJoCo **return** $\mathcal{Q}$ ▷ written to `data.qpos` by caller

该回调在 MuJoCo 主循环每帧被调用一次。对于典型设计（10 个关节），单次 `solve_motors()` 耗时约 $2\text{--}5\ \mu\text{s}$ ，加上限位处理和结果字典组装后总耗时约 $10\text{--}20\ \mu\text{s}$ ，远

小于 MuJoCo 引擎自身的步进耗时（约 1–5 ms），因此 SDAS 模型的引入不会对交互帧率造成可感知的影响。

SDAS 模型的总体算法流程

本节将前面各子节描述的模块组合成一个完整的算法流程，展示从 CAD 设计到实时关节角度生成的完整数据流与控制逻辑。算法 5 给出了这一全过程的伪代码。

Algorithm 5 SDAS 模型从 CAD 设计到实时关节角度生成的完整流程

Require: User completes hand design in CAD editor ▷ ===== Build Phase (once at startup) =====

- 1: User saves design as `design.ohd`
- 2: $design \leftarrow \text{OrigamiHandDesign.load}("design.ohd")$
- 3: $(\mathbf{R}_A, \mathbf{R}_B) \leftarrow \text{compute_R_one_sided}(design, \beta = 0.09)$
- 4: $model \leftarrow \text{SDASModel}(n, \mathbf{R}_A, \mathbf{R}_B, \mathbf{E}_{\text{vec}})$
- 5: Export URDF model from DXF path in $design$
- 6: $M \leftarrow \text{build_urdf_to_synergy_mapping}(urdf_path, design)$ ▷ ===== Run Phase (each frame) =====
- 7: $sim \leftarrow \text{MuJoCoSimulator}(urdf_path, synergy_callback, \dots)$
- 8: $prev_ctrl \leftarrow \mathbf{0}$
- 9: **while** sim is running **do**
- 10: $ctrl \leftarrow sim.get_ctrl_slider_values()$ ▷ Slider cross-coupling
- 11: **if** $\|ctrl_\theta - prev_ctrl_\theta\| > \|ctrl_\sigma - prev_ctrl_\sigma\|$ **then**
- 12: $\sigma \leftarrow (\theta_A + \theta_B)/2$; $\sigma_f \leftarrow (\theta_A - \theta_B)/2$
- 13: **else**
- 14: $\theta_A \leftarrow \max(0, \sigma + \sigma_f)$; $\theta_B \leftarrow \max(0, \sigma - \sigma_f)$
- 15: **end if**
- 16: $prev_ctrl \leftarrow ctrl$
- 17: $\mathbf{q} \leftarrow model.solve_motors(\theta_A, \theta_B)$
- 18: $\mathcal{Q} \leftarrow \text{clamp_and_assemble}(\mathbf{q}, M)$
- 19: $sim.set_joint_positions(\mathcal{Q})$
- 20: $sim.mj_forward()$
- 21: **end while**

算法 5 清晰地展示了三个关键设计特征。第一，构建阶段与运行阶段严格分离：所有矩阵预计算（ $\mathbf{S}_A^{(\text{schur})}$ 、 $\mathbf{S}_B^{(\text{schur})}$ 等）均在启动时一次性完成，运行时仅涉及标量与向量的线性组合，计算效率极高。第二，状态依赖的约束选择完全由 `solve_motors()` 内部自动处理，上层调用者无需关心当前处于单马达还是双马达模式。第三，滑块互联机制使得用户可以在 θ_A/θ_B 和 σ/σ_f 两种控制语言之间无缝切换，提高了交互的直观性。

从计算复杂度的角度来看，运行阶段的单帧计算包括一次 `solve_motors()` 调用（ $\mathcal{O}(n)$ ， n 为关节数）和一次关节限位遍历（ $\mathcal{O}(n)$ ），整体复杂度为 $\mathcal{O}(n)$ 。对于 $n = 10$ 的典型设计，单帧耗时在 10–20 μs 量级，足以支持 1 kHz 以上的控制频率。

小结

本节系统地介绍了 SDAS 模型的软件实现方法，涵盖从数据结构设计、传动矩阵计算、约束求解器实现到交互式仿真环境构建的完整技术路线。总结起来，SDAS 模型的软件实现具有以下关键特征。

基于 $\mathbf{R}_A$ 、 $\mathbf{R}_B$ 的自然非对称性取消了 $\mathbf{R}_f$ 概念。经典自适应协同模型需要引入额外的静摩擦冻结参数构建 $\mathbf{R}_f$ 矩阵，SDAS 模型以 Capstan 摩擦导致的路径依赖衰减自然产生 $\mathbf{R}_A \neq \mathbf{R}_B$ ，从而在无需额外参数的前提下实现二维协同空间控制。

状态依赖的约束选择策略。根据电机的激活状态自动切换求解公式：单马达拉动时使用单向公式，双马达同时拉动时使用 Schur 补公式。松弛钳位逻辑确保腱绳只能传递拉力这一物理事实被严格遵循，状态切换平滑自然。

完整的图形化设计入口。基于 PyQt5 的 Origami CAD 编辑器提供了可视化的折纸手设计环境，用户无需接触代码即可完成从折痕绘制到腱绳路径定义的全部设计工作。设计保存为 .ohd 文件后，后续的仿真流程可完全自动化。

与 MuJoCo 的高效集成。通过 URDF 导出、空间最近邻关节映射和回调函数机制，SDAS 求解器被嵌入 MuJoCo 物理引擎的主循环。四滑块互联动接口实现了 θ_A/θ_B 与 σ/σ_f 两种控制模式间的无缝切换。

计算效率方面，单次 `solve_motors()` 调用仅涉及若干标量与向量的线性组合操作，计算复杂度为 $\mathcal{O}(n)$ (n 为关节数)，单次求解时间在 $2\text{--}5\ \mu\text{s}$ 量级，完全满足 1 kHz 的实时控制要求。模块化的软件设计使得 SDAS 模型可同时服务于运动学交互仿真和动力学数值仿真两种应用场景，为后续的仿真验证与实验研究奠定了坚实的软件基础。